DYNAMIC PROGRAMMING INTRODUCTION

DATA STRUCTURES AND ALGORITHMS 2

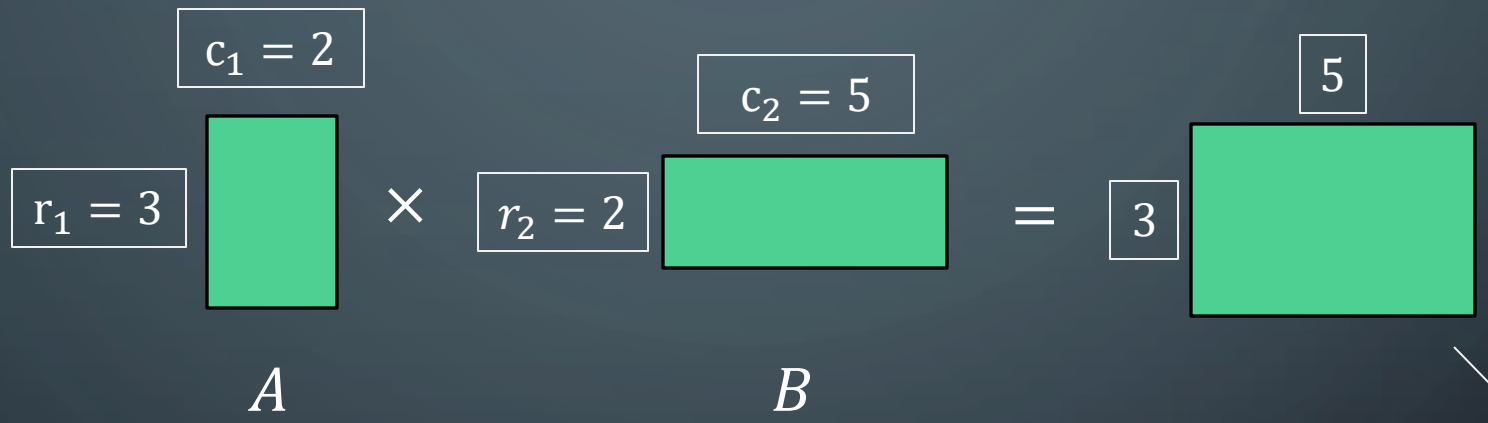
MARK FLORYAN

DYNAMIC PROGRAMMING AND GREEDY APPROACH

- TOPICS:
  - Intro to Dynamic Programming
  - Memoization
  - Example DP Problem:
    - Log Cutting

MOTIVATING EXAMPLE

How many scalar multiplications are required to multiply matrices A and B in this example?



$r_1 c_2$ elements in result matrix to compute

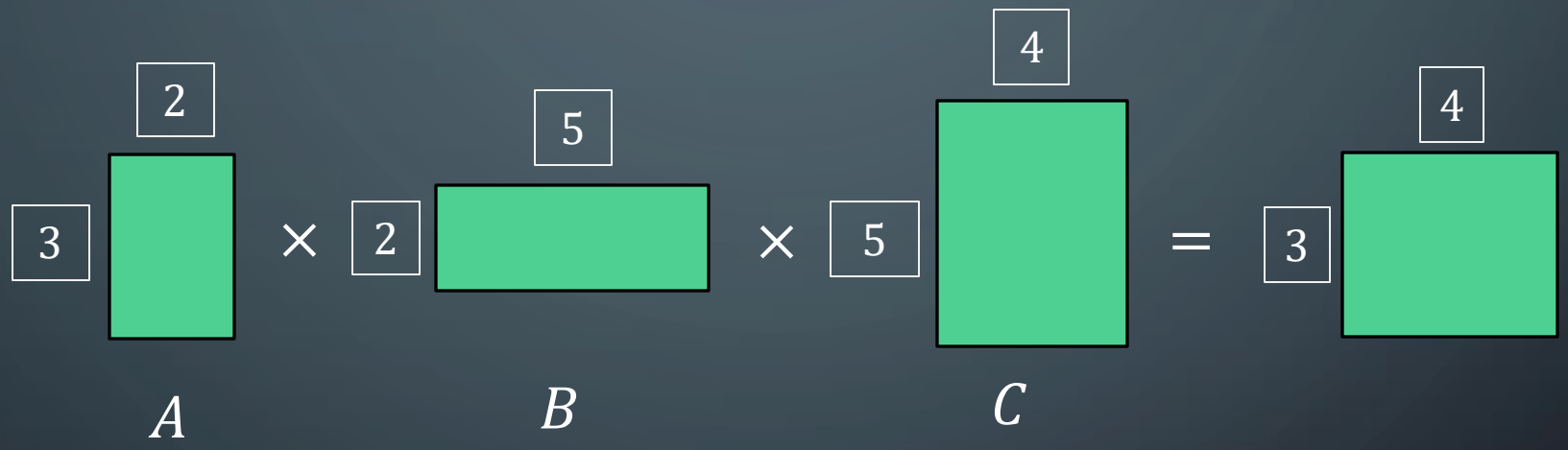
c_1 multiplications per element in result

Total is:

$r_1 c_2 c_1 = (3)(5)(2) = 30$

TRICKIER EXAMPLE

How many scalar multiplications are required to multiply matrices A, B and C in this example?



Same formula:

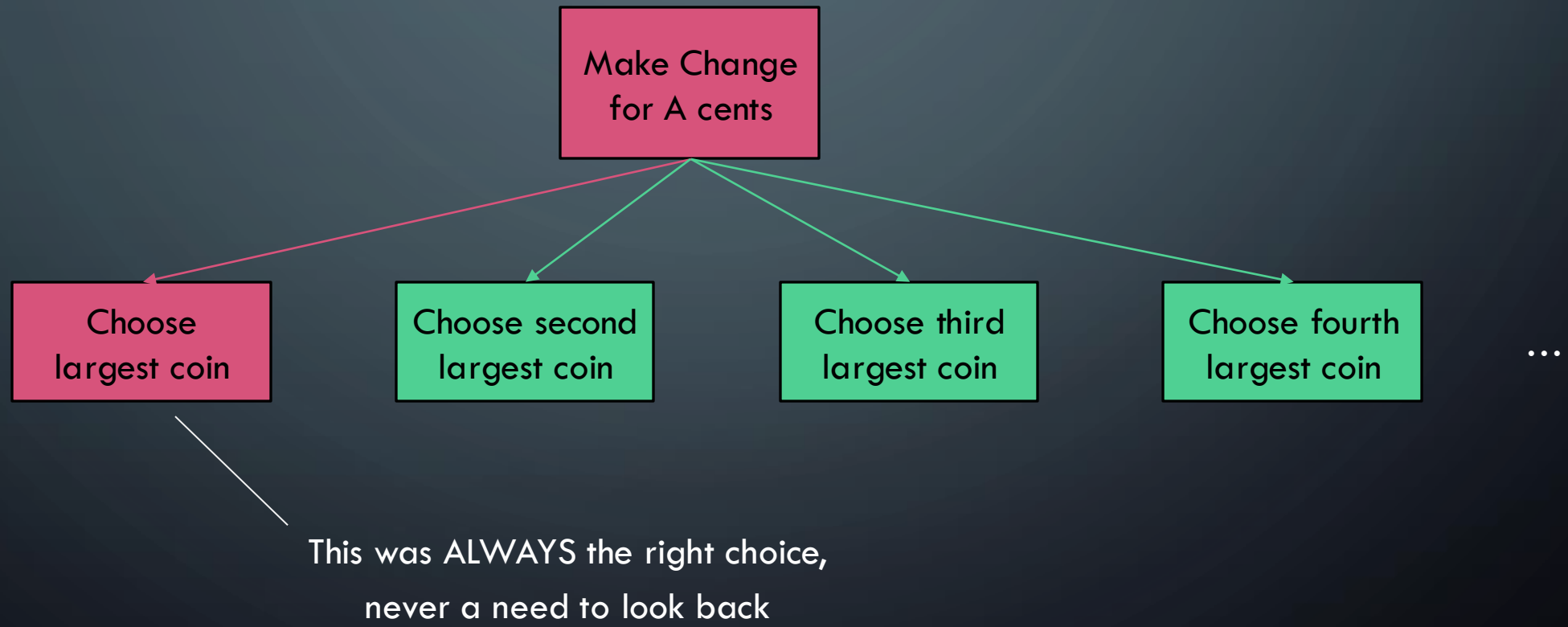
$$(r_1 c_1 c_2) + (r_1 c_2 c_3) = [(3)(2)(5)] + [(3)(5)(4)] = 90$$

Can actually be done in just 64!

Think about why this might be?

DYNAMIC PROGRAMMING AND GREEDY APPROACH

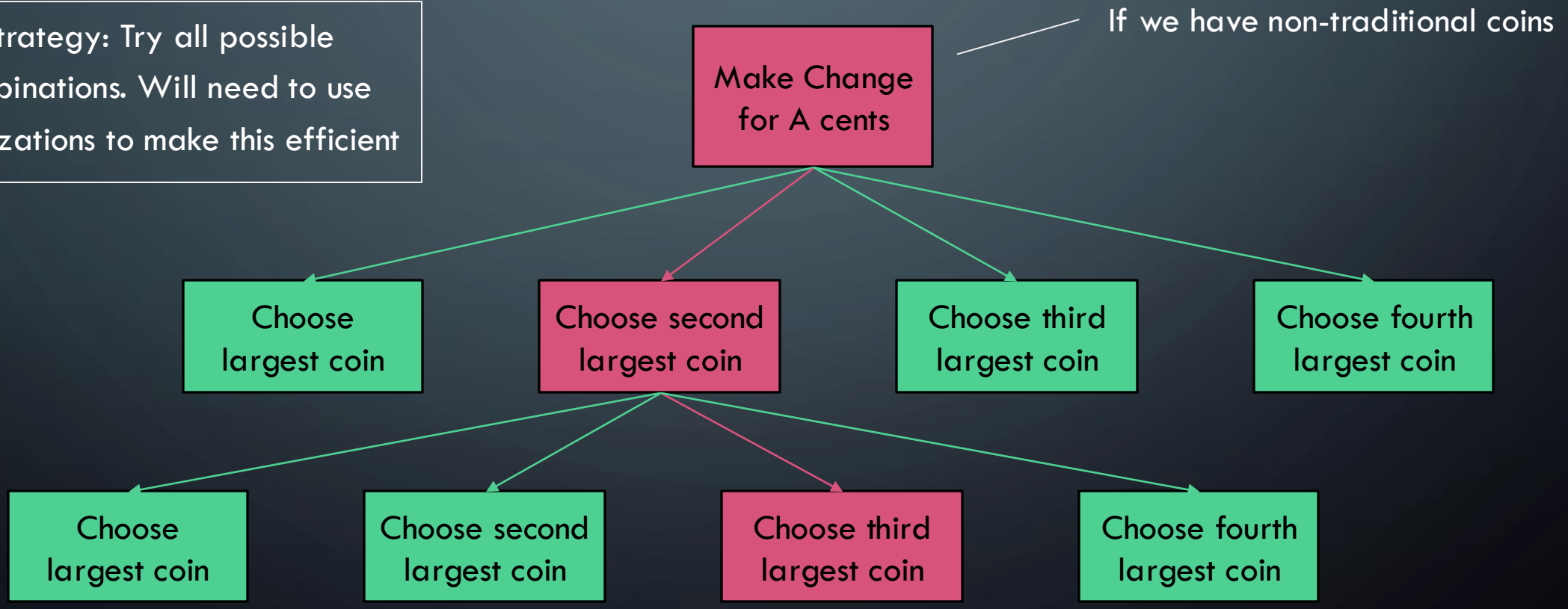
With ***Greedy Algorithms***, problem had ***optimal substructure*** and there was a known choice to make at each step of the problem that guaranteed an optimal solution



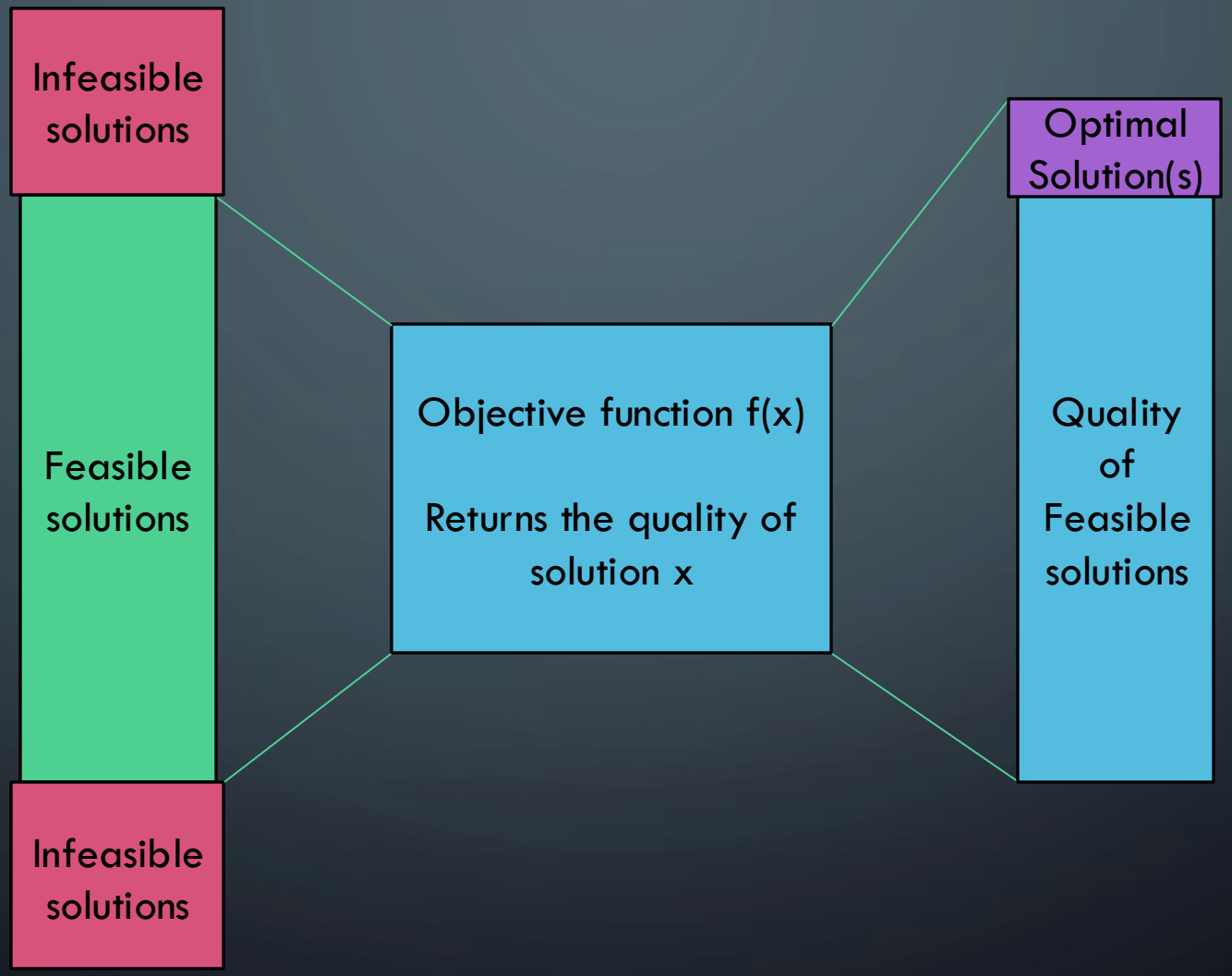
DYNAMIC PROGRAMMING AND GREEDY APPROACH

With ***Dynamic Programming***, problem has ***optimal substructure*** but we don't know which choice at each step will lead to optimal solution

Strategy: Try all possible combinations. Will need to use optimizations to make this efficient



REMINDER: OPTIMIZATION PROBLEMS



MEMOIZATION

REMEMBER FIBONACCI NUMBERS?

Formula: $f(n) = f(n-1) + f(n-2)$

```
long fib(int n) {
        assert(n >= 0);
        if ( n == 0 ) return 0;
        if ( n == 1 ) return 1;
        return fib(n-1) + fib(n-2);
}
```

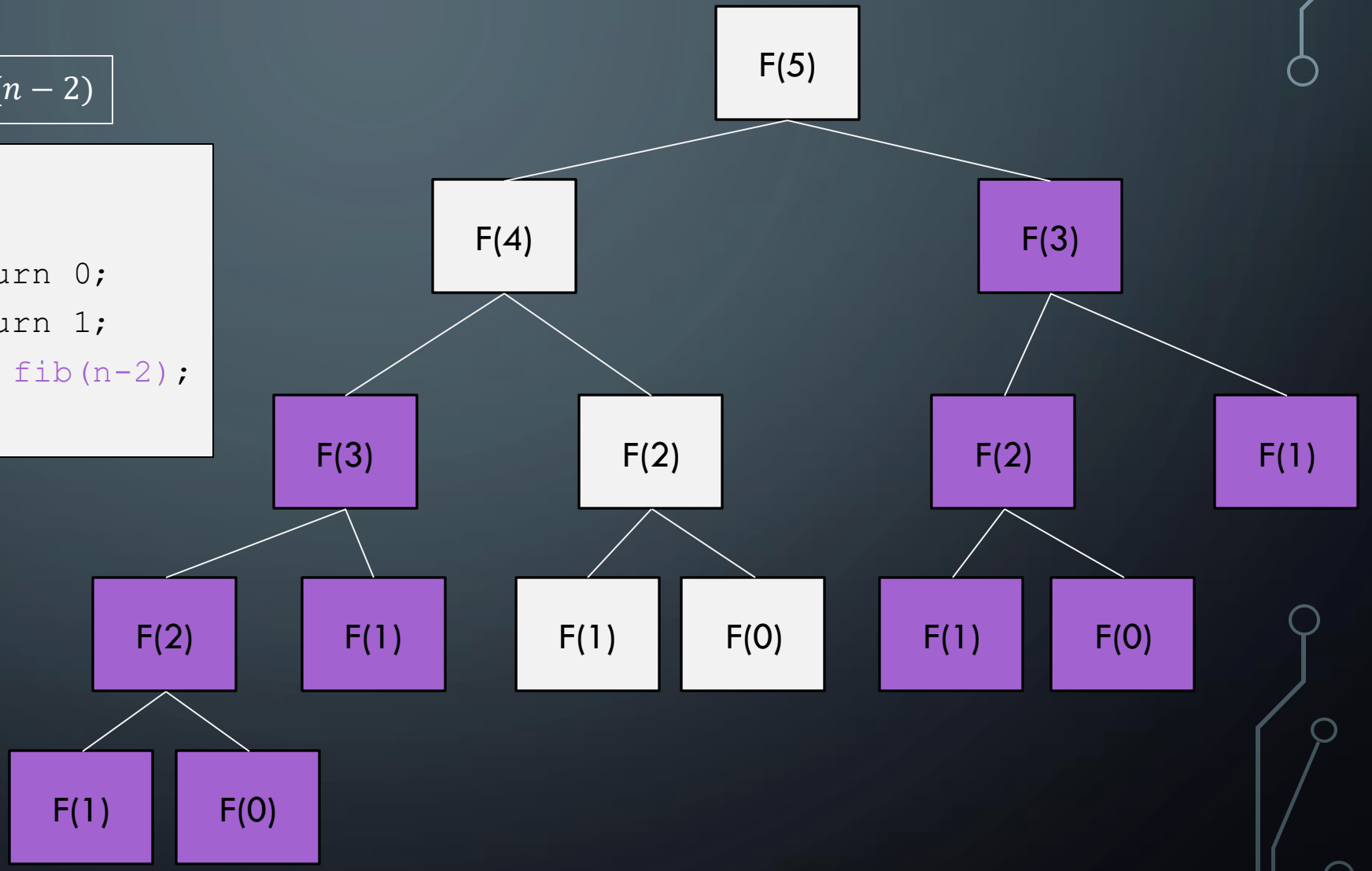
This is a BAD implementation! Repeats function calls repeatedly and runtime is obscenely exponential.

REMEMBER FIBONACCI NUMBERS?

Formula: $f(n) = f(n-1) + f(n-2)$

```
long fib(int n) {
        assert(n >= 0);
        if ( n == 0 ) return 0;
        if ( n == 1 ) return 1;
        return fib(n-1) + fib(n-2);
}
```

Entire sub-trees are repeated!!

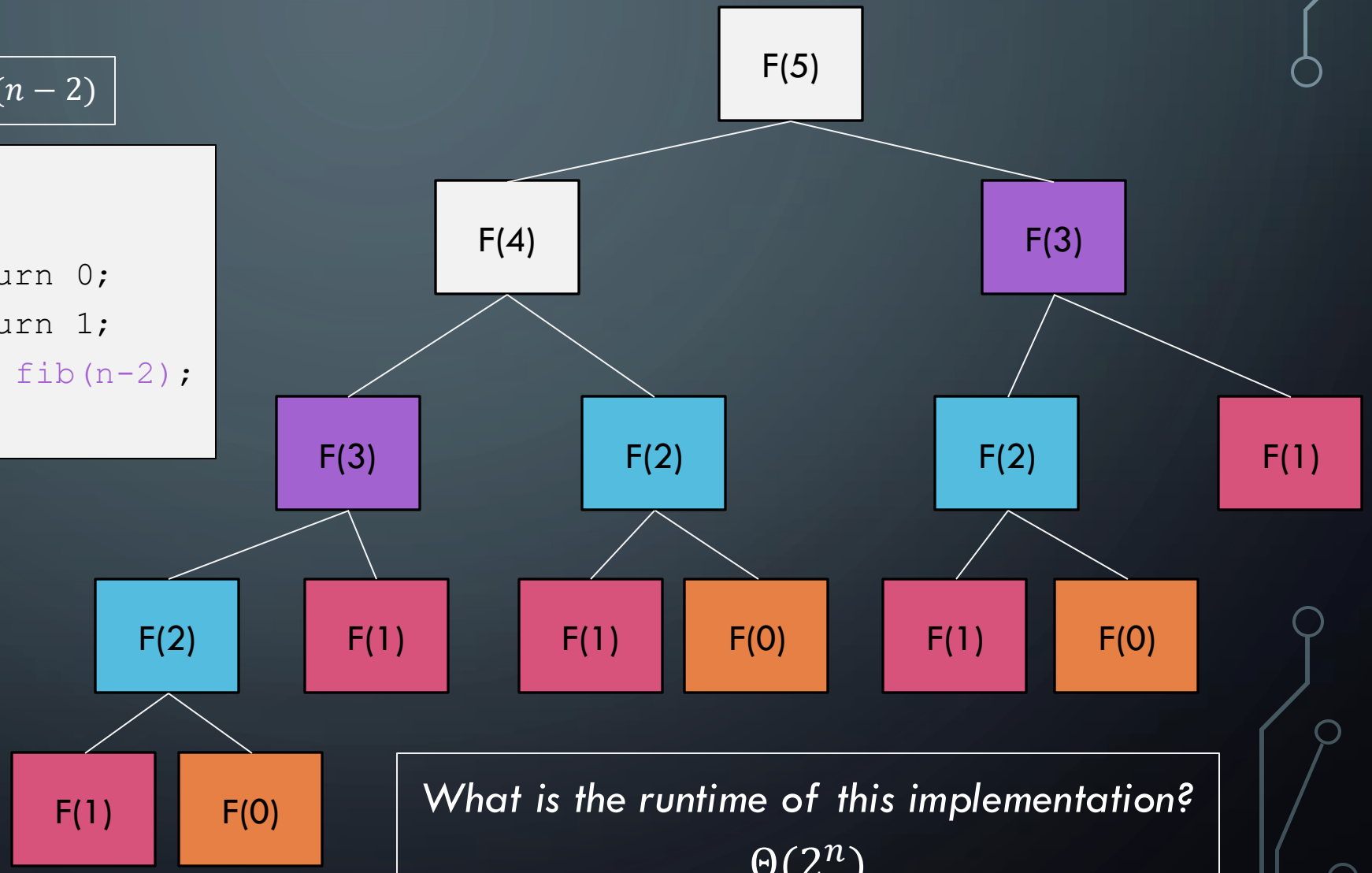


REMEMBER FIBONACCI NUMBERS?

Formula: $f(n) = f(n-1) + f(n-2)$

```
long fib(int n) {
        assert(n >= 0);
        if ( n == 0 ) return 0;
        if ( n == 1 ) return 1;
        return fib(n-1) + fib(n-2);
}
```

Look at how many recursive calls are repeated!



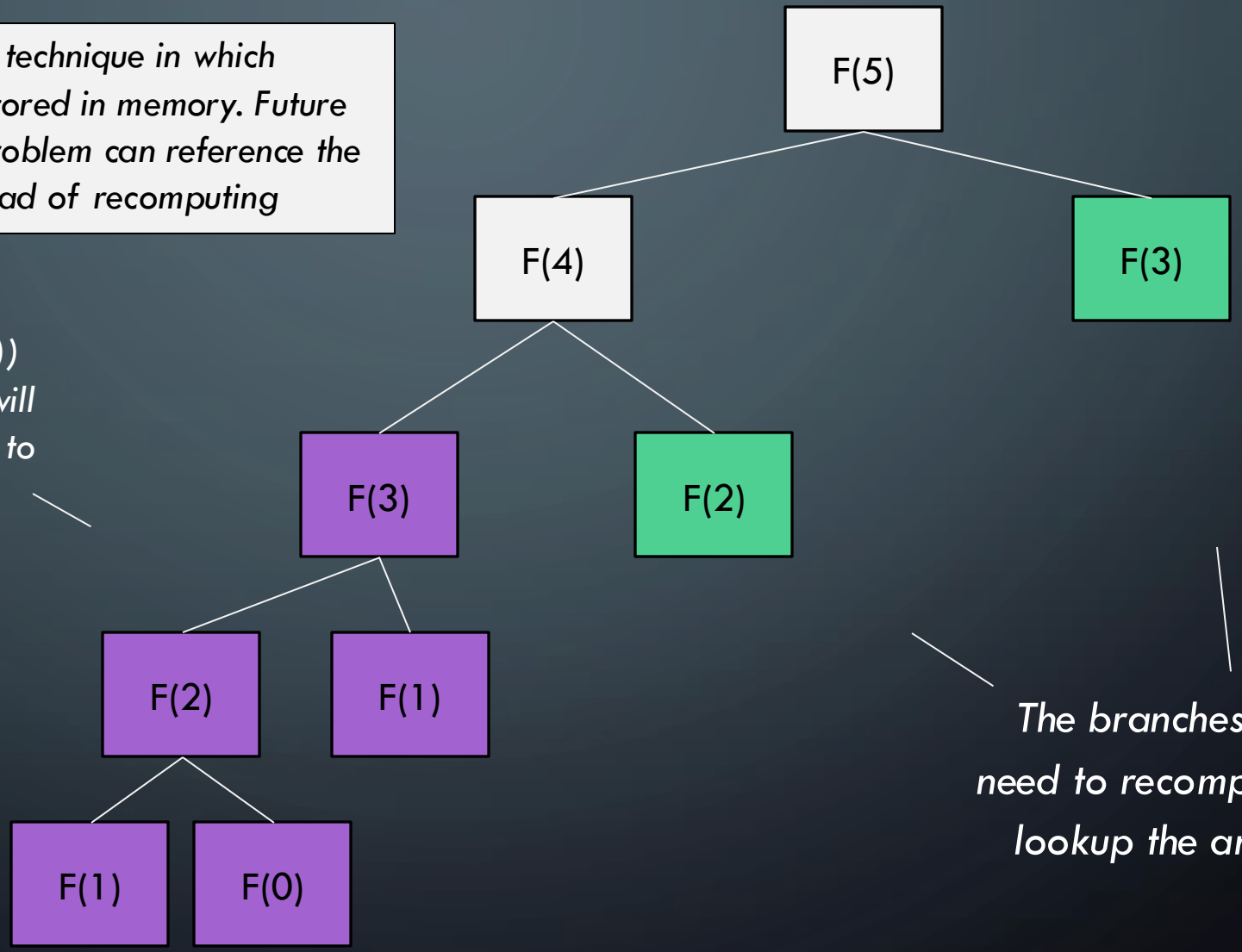
What is the runtime of this implementation?

$\Theta(2^n)$

MEMOIZATION

Memoization: *A programming technique in which solutions to subproblems are stored in memory. Future invocations of that same subproblem can reference the previously stored solution instead of recomputing*

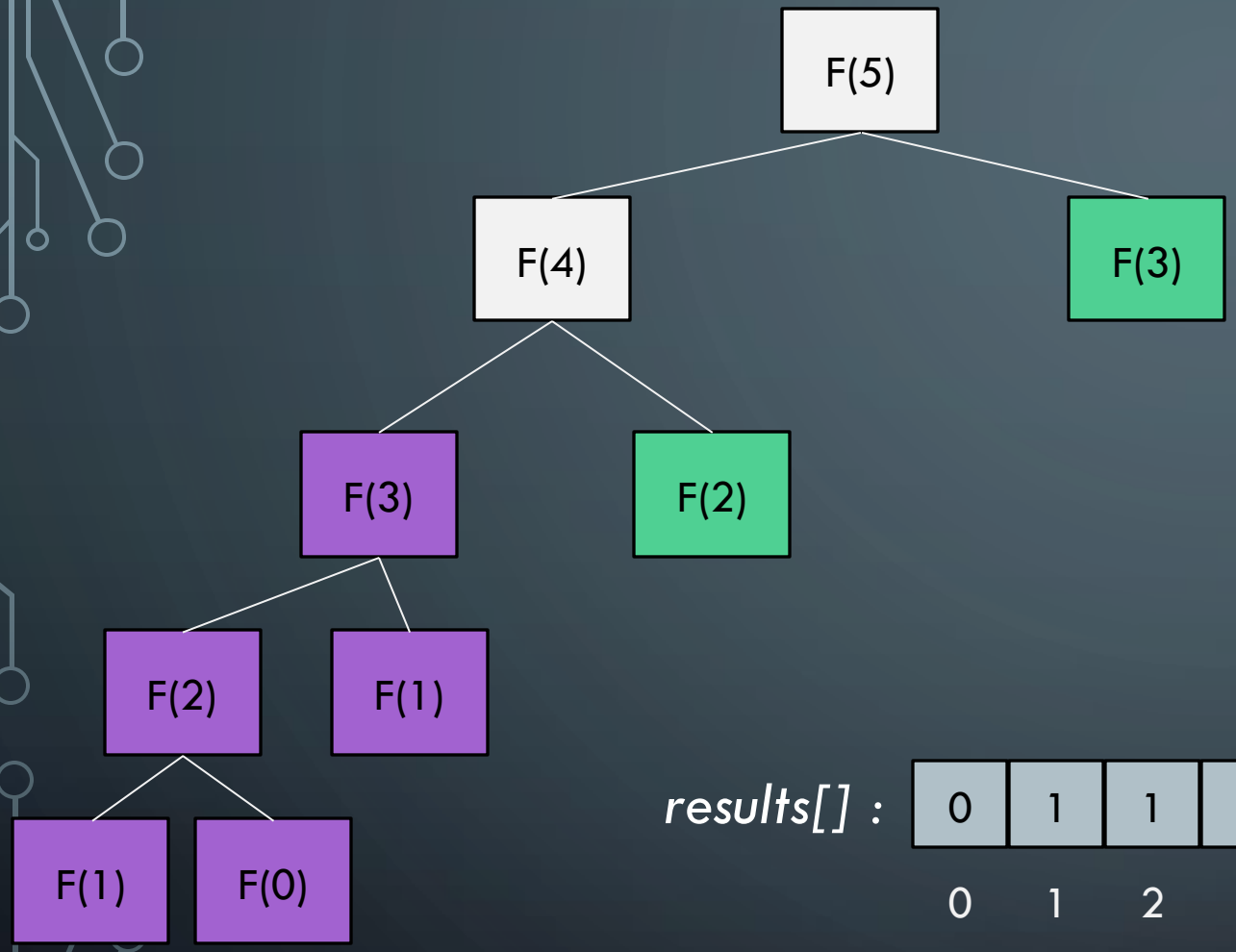
When you call a subproblem (e.g., f(3)) the first time, compute the answer! This will computer the answer and store solutions to f(3), f(2), f(1), and f(0) in an array!



The branches don't need to recompute, just lookup the answer!

MEMOIZATION

Key Idea: *You need to be able to lookup a solution to a subproblem in constant time!*



Value is -1 if that subproblem not solved yet, otherwise table contains the solution!

Last cell is the fib number we want to calculate! Here it is 11

results[] :

0	1	2	3	4	5	6	7	8	9	10	11
0	1	1	2	-1	-1	-1	-1	-1	-1	-1	-1

Memoization table. Stores solutions to subproblems!

MEMOIZATION AND FIBONACCI

```
long results[]

long fib(int n):
   results = {-1}*n
   return fibRec(n)

long fibRec(int n):
   if ( results[n] != -1 ):
      return results[n]

   long val
   if ( n == 0 || n ==1 ):
      val = n
   else
      val = fibRec(n-1)+ fibRec(n-2)


   results[n] = val
   return val
```

MEMOIZATION AND FIBONACCI

```
long results[]

long fib(int n):
   results = {-1}*n
   return fibRec(n)

long fibRec(int n):

   if ( results[n] != -1 ):
      return results[n]

   long val
   if ( n == 0 || n ==1 ):
      val = n
   else
      val = fibRec(n-1)+ fibRec(n-2)


   results[n] = val
   return val
```

Make array filled with -1 everywhere and kick off the recursion!

If subproblem already solved, just return the answer and don't do any recursion!

Normal recursive implementation

Before returning, store the result in results so other subproblem calls can use it!

MEMOIZATION AND FIBONACCI

```
long results[]

long fib(int n):
   results = {-1}*n
   return fibRec(n)

long fibRec(int n):
   if ( results[n] != -1 ):
      return results[n]

   long val
   if ( n == 0 || n ==1 ):
      val = n
   else
      val = fibRec(n-1)+ fibRec(n-2)


   results[n] = val
   return val
```

Runtime: $\Theta(n)$

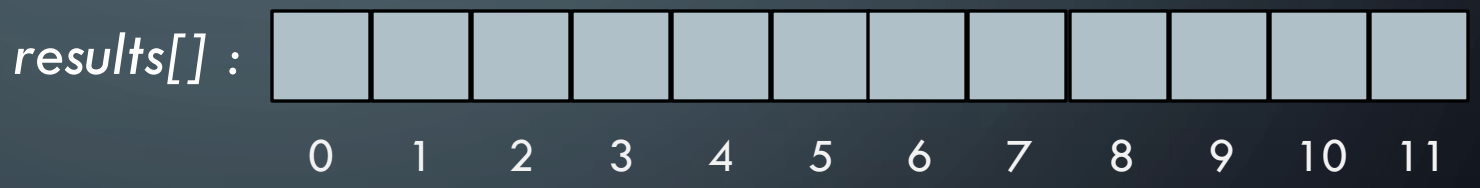
Why?: *Because each cell in results is computed exactly once. Future calls return in constant time.*

This is called ***Top-Down Dynamic Programming***

MEMOIZATION AND FIBONACCI

```
long results[]

long fib(int n):
   results = {-1}*n
   return fibRec(n)

long fibRec(int n):
   if ( results[n] != -1 ):
      return results[n]

   long val
   if ( n == 0 || n ==1 ):
      val = n
   else
      val = fibRec(n-1)+ fibRec(n-2)


   results[n] = val
   return val
```

*Why is this called **Top-Down Dynamic Programming**? Let's step through it!*



BOTTOM-UP DYNAMIC PROGRAMMING

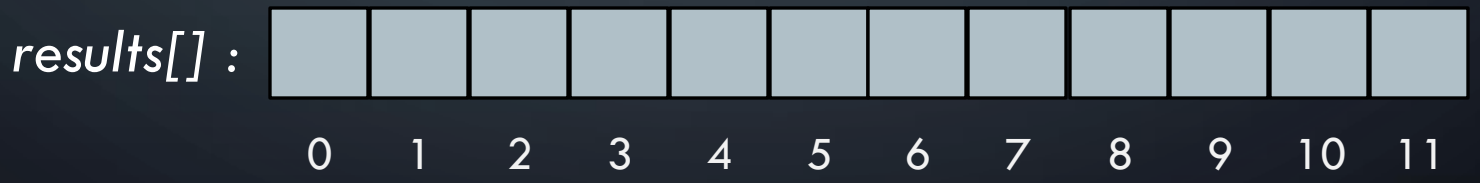
Bottom-Up Dynamic Programming: Involves solving the base cases first and then filling in the solutions in (usually) increasing order. This is typically NOT recursive

BOTTOM-UP FIBONACCI

```
fib(int n):
    long result[n+1]
    result[0] = 0; result[1] = 1

    for i = 2 to n:
        result[i] = result[i-1] + result[i-2]

    return result[n]
```

BOTTOM-UP FIBONACCI

```
fib(int n):
    long result[n+1]
    result[0] = 0
    result[1] = 1

    for i = 2 to n:
        result[i] = result[i-1] + result[i-2]

    return result[n]
```



DYNAMIC PROGRAMMING AND LOG CUTTING

PROCESS FOR DYNAMIC PROGRAMMING

Step 1:

Recognize what the ***sub-problems*** are

Step 2:

Identify the recursive structure of the problem in terms of its sub-problems (At the top level, what is the "last thing" done?)

Step 3:

Formulate a ***data structure (array, table)*** that can look-up solution to any sub-problem in ***constant time***

Step 4:

Develop an algorithm that ***loops through data structure*** solving each sub-problem one at a time (either bottom-up or top-down)

LOG CUTTING

Problem Statement: Given a log of length n, and a list (of length n) of prices P (where $P[i]$ is the price of a cut of size i). Find the best way to cut the log to maximize our profit.

Length:	0	1	2	3	4	5	6	7	8	9	10
Price:	0	1	5	8	9	10	17	17	20	24	30

Select a list of lengths $l_1, \ldots, l_k$ such that:
$\sum l_i = n$ and Maximize $\sum P[\ell_i]$

Brute Force: $O(2^n)$

PROCESS FOR DYNAMIC PROGRAMMING

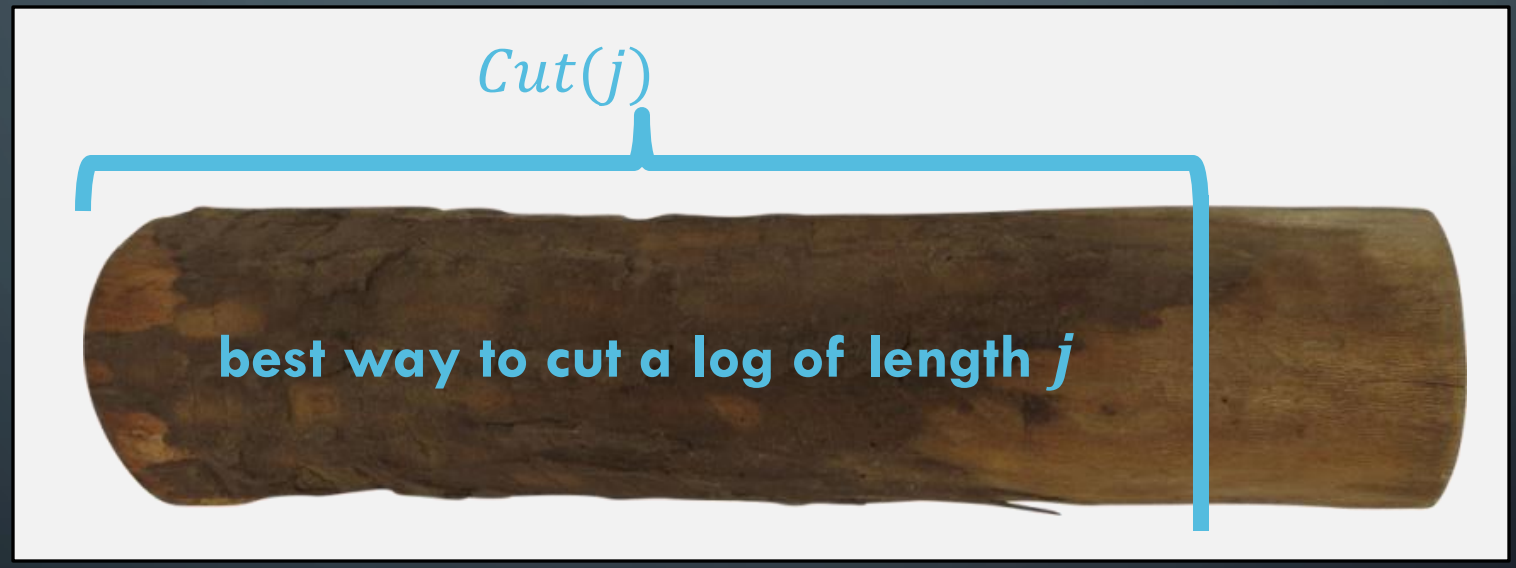
Step 1:

Recognize what the ***sub-problems*** are

1. IDENTIFY SUB-PROBLEMS

$P[i] =$ value of a cut of length i

$Cut(j) =$ value of best way to cut a log of length j



Notice that cut(n) ranges from cut(0) all the way to cut(n)

PROCESS FOR DYNAMIC PROGRAMMING

Step 1:

Recognize what the ***sub-problems*** are

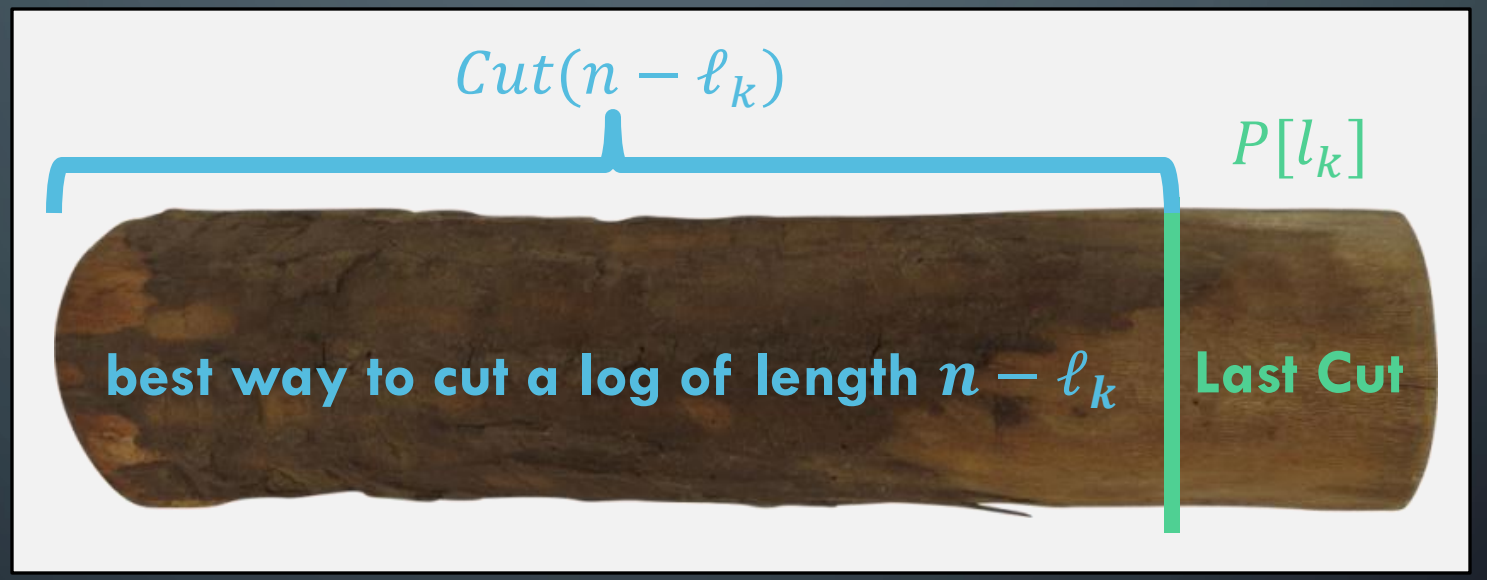
Step 2:

Identify the recursive structure of the problem in terms of its sub-problems (At the top level, what is the "last thing" done?)

2. IDENTIFY RECURSIVE STRUCTURE

$P[i]$ = value of a cut of length i

$Cut(n)$ = value of best way to cut a log of length n



Key Idea: *Somewhere the very last cut was made. The solution is the value you get for the last cut plus the optimal way to cut up the rest of the log*

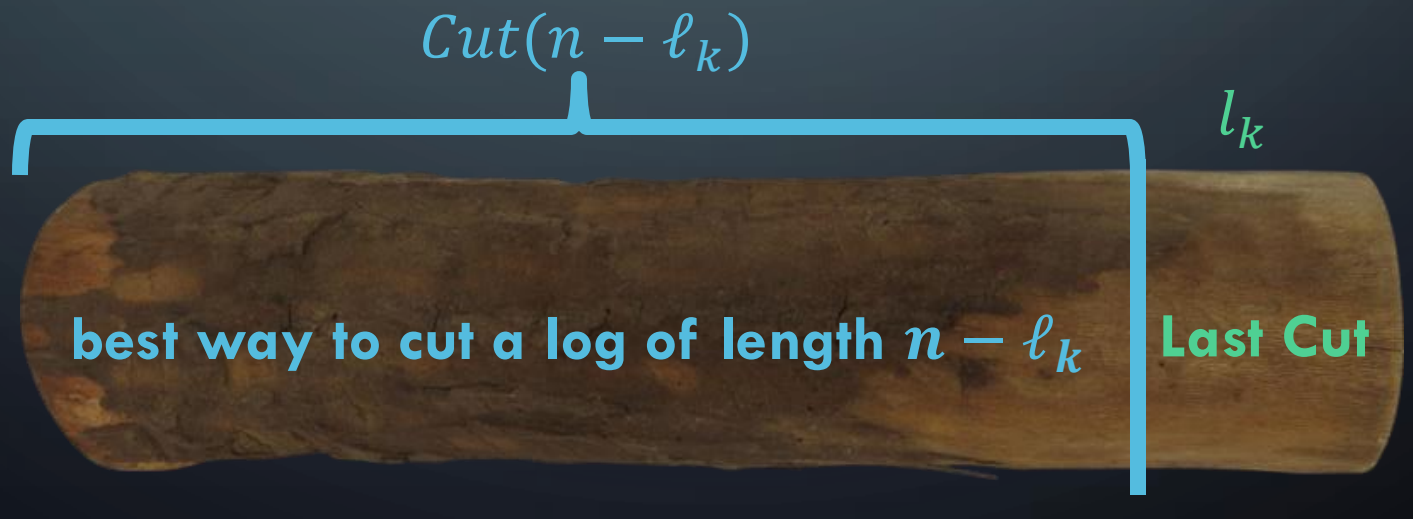
2. IDENTIFY RECURSIVE STRUCTURE

$P[i] =$ value of a cut of length i

$Cut(n) =$ value of best way to cut a log of length n

$$Cut(n) = \max \begin{cases} Cut(n-1) + P[1] \\ Cut(n-2) + P[2] \\ \dots \\ Cut(0) + P[n] \end{cases}$$

Try every possible place to put the last cut!



PROCESS FOR DYNAMIC PROGRAMMING

Step 1:

Recognize what the ***sub-problems*** are

Step 2:

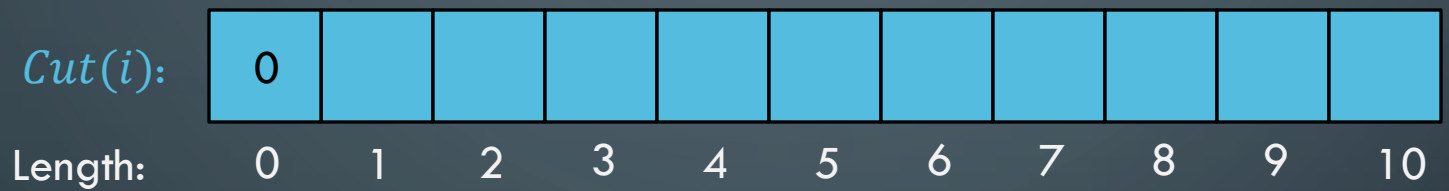
Identify the recursive structure of the problem in terms of its sub-problems (At the top level, what is the "last thing" done?)

Step 3:

Formulate a ***data structure (array, table)*** that can look-up solution to any sub-problem in ***constant time***

3. FORMULATE A LOOK-UP DATA STRUCTURE

Cell 0 is a base case cell. By definition the profit is 0



Each cell Cut[i] stores the optimal profit you can get by cutting up a log of size i

For example, Cut[5] would store the optimal profit from cutting up a log of size 5

The solution we are looking for (n) will be in this last cell!

PROCESS FOR DYNAMIC PROGRAMMING

Step 1:

Recognize what the ***sub-problems*** are

Step 2:

Identify the recursive structure of the problem in terms of its sub-problems (At the top level, what is the "last thing" done?)

Step 3:

Formulate a ***data structure (array, table)*** that can look-up solution to any sub-problem in ***constant time***

Step 4:

Develop an algorithm that ***loops through data structure*** solving each sub-problem one at a time (either bottom-up or top-down)

3. ALGORITHM TO FILL IN SOLUTIONS

Solve smallest sub-problem first

Price:	1	5	8	9	10	17	17	20	24	30
Length:	1	2	3	4	5	6	7	8	9	10

Cut:	0									
Length:	0	1	2	3	4	5	6	7	8	9	10

```
CutProfit(n):
    Cut[n] = [...]
    Cut[0] = 0

    ...
```

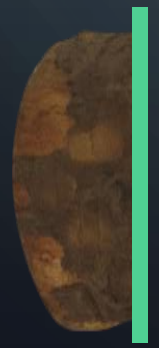
3. ALGORITHM TO FILL IN SOLUTIONS

Then the next largest subproblem

Length:	1	2	3	4	5	6	7	8	9	10
Price:	1	5	8	9	10	17	17	20	24	30

Length:	0	1	2	3	4	5	6	7	8	9	10
Cut:	0	1									

```
CutProfit(n):
    Cut[n] = [...]
    Cut[0] = 0

    for i = 1 to n:
        ?????
```



$$Cut[1] = Cut[0] + P[1] = 1$$

3. ALGORITHM TO FILL IN SOLUTIONS

Then the next largest subproblem

Length:	1	2	3	4	5	6	7	8	9	10
Price:	1	5	8	9	10	17	17	20	24	30

Length:	0	1	2	3	4	5	6	7	8	9	10
Cut:	0	1	5								

```
CutProfit(n):
    Cut[n] = [...]
    Cut[0] = 0

    for i = 1 to n:
        ?????
```

$$Cut[2] = \max \begin{cases} Cut[1] + P[1] = 2 \\ Cut[0] + P[2] = 5 \end{cases}$$

3. ALGORITHM TO FILL IN SOLUTIONS

Then the next largest subproblem

Length:	1	2	3	4	5	6	7	8	9	10
Price:	1	5	8	9	10	17	17	20	24	30

Length:	0	1	2	3	4	5	6	7	8	9	10
Cut:	0	1	5	8							

```
CutProfit(n):
  Cut[n] = […]
  Cut[0] = 0

  for i = 1 to n:
    best = 0
    for j = 1 to i:
      best = max(Cut[i-j]+P[j]
    Cut[i] = best
```

$$Cut[3] = \max \begin{cases} Cut[2] + P[1] = 6 \\ Cut[1] + P[2] = 6 \\ Cut[0] + P[3] = 8 \end{cases}$$

3. ALGORITHM TO FILL IN SOLUTIONS

Then the next largest subproblem

Length:	1	2	3	4	5	6	7	8	9	10
Price:	1	5	8	9	10	17	17	20	24	30

Length:	0	1	2	3	4	5	6	7	8	9	10
Cut:	0	1	5	8	10						

```
CutProfit(n):
  Cut[n] = […]
  Cut[0] = 0

  for i = 1 to n:
    best = 0
    for j = 1 to i:
      best = max(Cut[i-j]+P[j]
    Cut[i] = best
```

$$Cut[3] = \max \begin{cases} Cut[3] + P[1] = 9 \\ Cut[2] + P[2] = 10 \\ Cut[1] + P[3] = 9 \\ Cut[0] + P[4] = 9 \end{cases}$$

3. ALGORITHM TO FILL IN SOLUTIONS

Then the next largest subproblem

Price:	1	5	8	9	10	17	17	20	24	30
Length:	1	2	3	4	5	6	7	8	9	10

Cut:	0	1	5	8	10	13	17	18	22	25	30
Length:	0	1	2	3	4	5	6	7	8	9	10

Solution is here in the last cell, so just return it!

```
CutProfit(n):
  Cut[] = new [n+1]
  Cut[0] = 0

  for i = 1 to n:
    best = 0
    for j = 1 to i:
      best = max(Cut[i-j]+P[j]
    Cut[i] = best

  return Cut[n]
```

3. ALGORITHM TO FILL IN SOLUTIONS

Then the next largest subproblem

Length:	0	1	2	3	4	5	6	7	8	9	10
Cut:	0	1	5	8	10	13	17	18	22	25	30

```
CutProfit(n):
  Cut[] = new [n+1]
  Cut[0] = 0

  for i = 1 to n:
    best = 0
    for j = 1 to i:
      best = max(Cut[i-j]+P[j]
    Cut[i] = best

  return Cut[n]
```

Runtime is $\Theta(n^2)$

PROCESS FOR DYNAMIC PROGRAMMING

Step 1:

Recognize what the ***sub-problems*** are

Step 2:

Identify the recursive structure of the problem in terms of its sub-problems (At the top level, what is the "last thing" done?)

Step 3:

Formulate a ***data structure (array, table)*** that can look-up solution to any sub-problem in ***constant time***

Step 4:

Develop an algorithm that ***loops through data structure*** solving each sub-problem one at a time (either bottom-up or top-down)

HOW TO FIND THE ACTUAL CUTS

EXTRA: FINDING THE ACTUAL SOLUTION

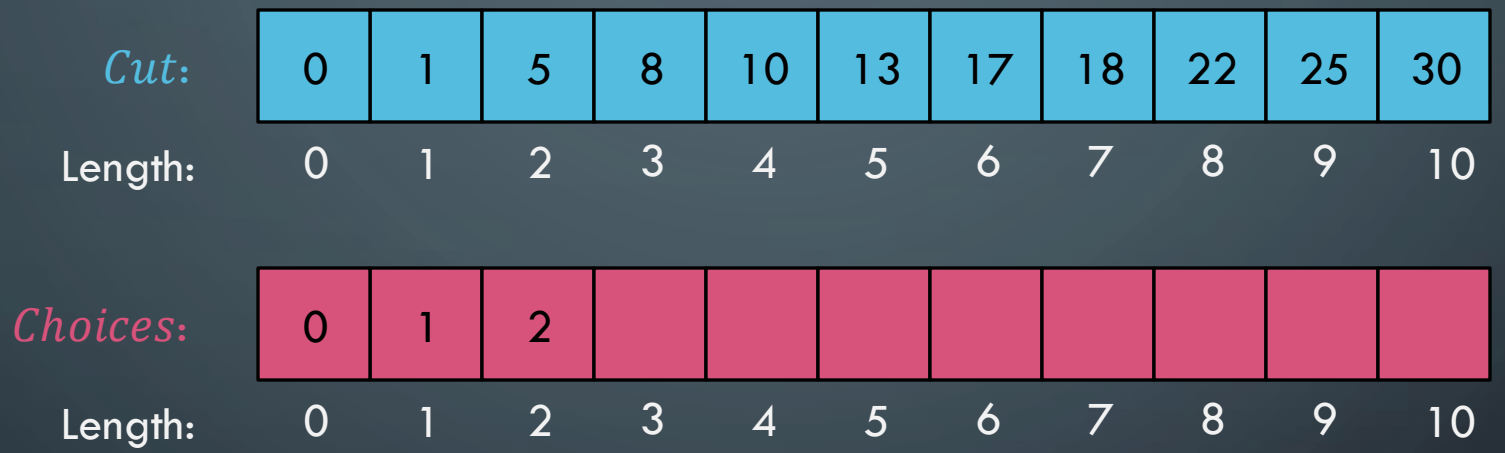
This gives the value of the solution, what about the actual list of where to cut the log?

Cut:	0	1	5	8	10	13	17	18	22	25	30
Length:	0	1	2	3	4	5	6	7	8	9	10

Idea: Keep track of where the last cut was for each subproblem!

EXTRA: FINDING THE ACTUAL SOLUTION

This gives the value of the solution, what about the actual list of where to cut the log?



$$Cut[2] = \max \begin{cases} Cut[1] + P[1] = 2 \\ \boldsymbol{Cut[0] + P[2] = 5} \end{cases}$$

Idea: Keep track of where the last cut was for each subproblem!

e.g., Choices[2]=2 because last cut was a piece of size 2

EXTRA: FINDING THE ACTUAL SOLUTION

So in this case, the optimal solution is to just sell the entire log of size 10

EXTRA: FINDING THE ACTUAL SOLUTION

Cut:	0	1	5	8	10	13	17	18	22	25	30
Length:	0	1	2	3	4	5	6	7	8	9	10

Choices:	0	1	2	3	2	2	6	1	2	3	10
Length:	0	1	2	3	4	5	6	7	8	9	10

```
CutProfit(n):
  Cut[] = new [n+1]
  Choices[] = new [n+1]
  Cut[0] = 0
  Choices[0] = 0

  for i = 1 to n:
    best = 0
    for j = 1 to i:
      if best < Cut[i-j]+P[j]:
        best = Cut[i-j]+P[j]
        Choices[i] = j
    Cut[i] = best

  return Cut[n]
```

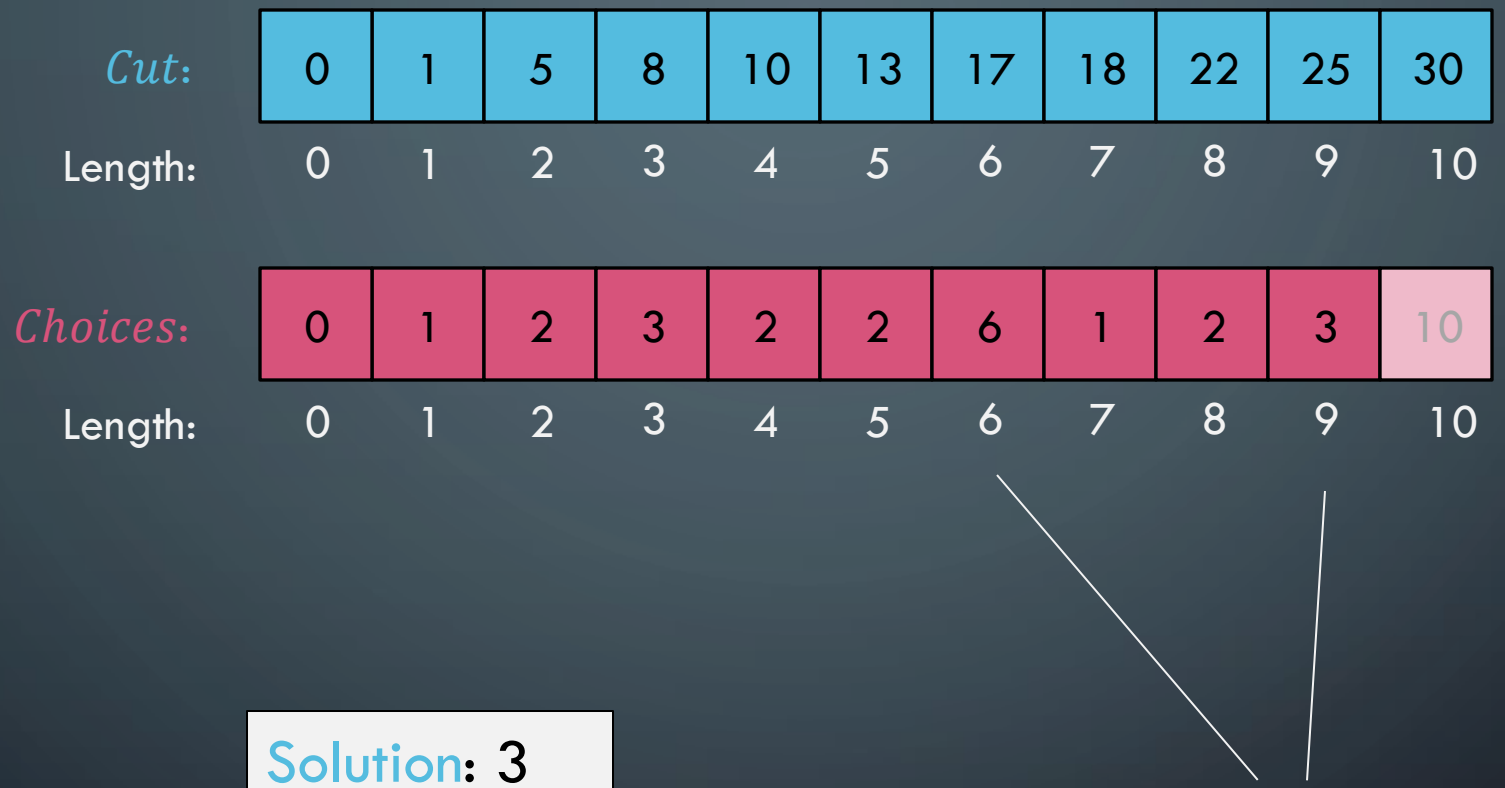
EXTRA: FINDING THE ACTUAL SOLUTION

Cut:	0	1	5	8	10	13	17	18	22	25	30
Length:	0	1	2	3	4	5	6	7	8	9	10

Choices:	0	1	2	3	2	2	6	1	2	3	10
Length:	0	1	2	3	4	5	6	7	8	9	10

Let's use this choices array to figure out how to achieve a profit of 25 with a log of size 9?

EXTRA: FINDING THE ACTUAL SOLUTION



Solution: 3
Left: 6

This 3 means that we cut off and sell a log piece of size 3

Log of size 6 left so backtrack to index 6

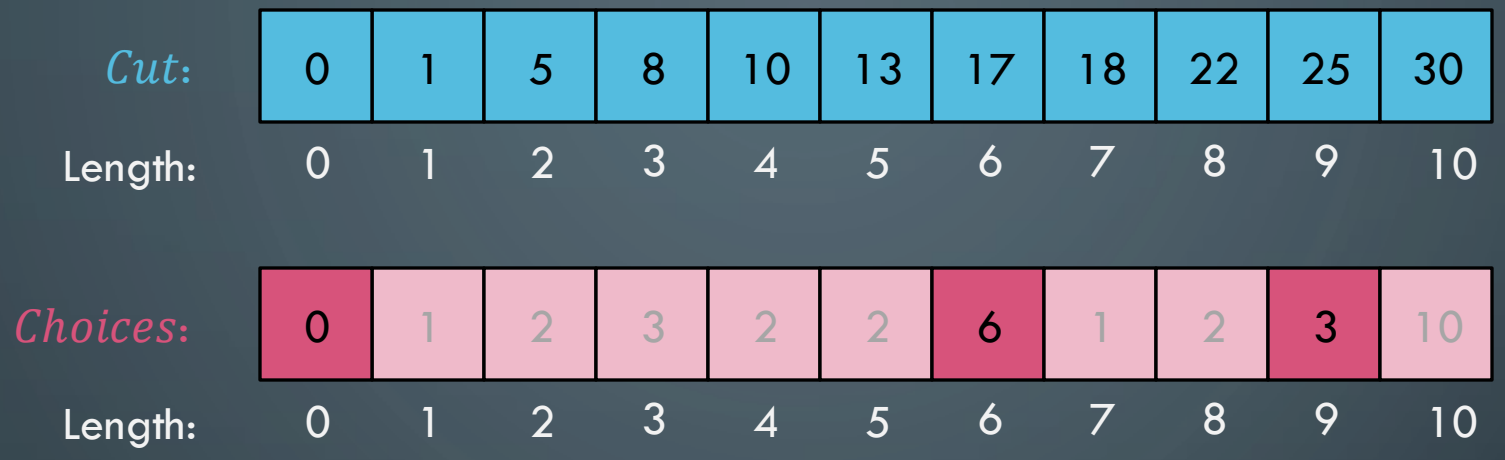
EXTRA: FINDING THE ACTUAL SOLUTION

Solution: 3 6
Left: 0

This 6 means that we cut off and sell a log piece of size 6

Log of size 0 left so backtrack to index 0

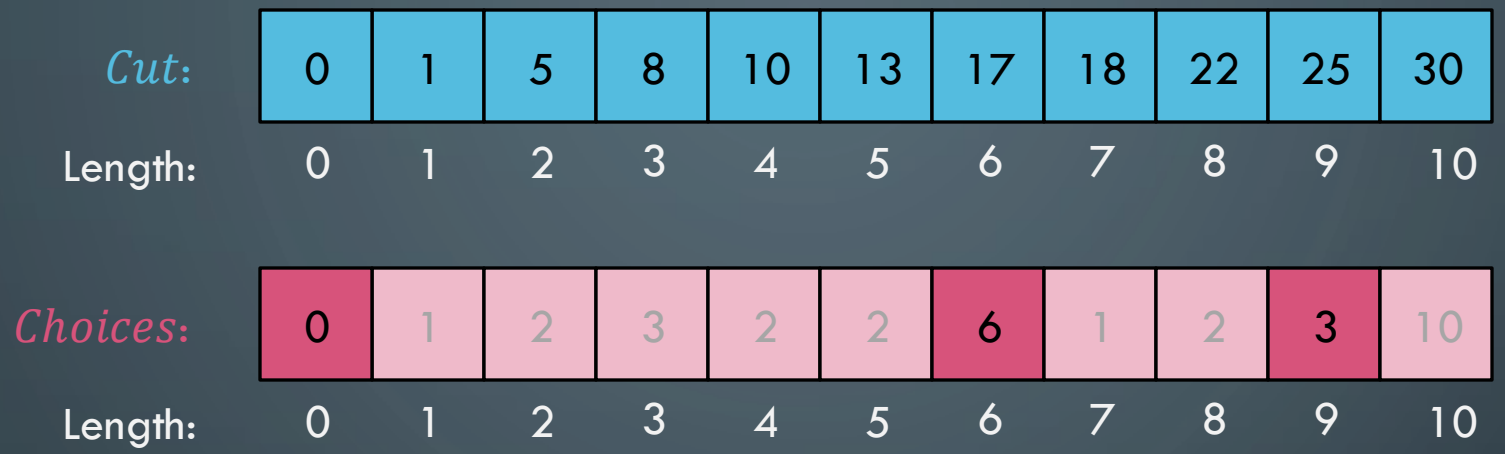
EXTRA: FINDING THE ACTUAL SOLUTION



Solution: 3 6

Once we hit index 0 we are done. Solution is to split log into sizes 3 and 6 and sell those two pieces!

EXTRA: FINDING THE ACTUAL SOLUTION



Solution: 3 6

```
BackTrack(Choices):
  i = n
  while i > 0:
    print Choices[i]
    i = i - Choices[i]
```

With dynamic programming, it is VERY common to have the array store the value of solutions only and then use backtracking to compute the solution itself!

WHAT DID WE LEARN?

- What is dynamic programming? General Approach
- First Example: Log cutting